

Mini Project 1

SGAIM

Construção de um GPT from scratch

Bruno Rodrigues nº 52323
Mestrado em Engenharia Informática e Multimédia
ISEL - 2024/2025

Índice

Exercício 1: The Autograd Engine	4
Objetivo	4
Implementação	4
Operações aritméticas (add, mul)	4
Operações unarias (pow, log, exp, relu)	4
Backward pass	4
Think: $c = a * b + a$ ($a=2, b=3$)	4
Tokenizer	4
Output	4
Exercício 2: Neural Network Primitives	6
Objetivo	6
Implementação	6
Linear	6
Softmax	6
RMSNorm	6
Output	6
Reflect	6
Exercício 3: Self-Attention	7
Objetivo	7
Implementação	7
Projecoes Q, K, V	7
Scores de atenção	7
Weighted sum	7
Think	7
Output	7
Exercício 4: The GPT Model	8
Objetivo	8
Implementação	8
Multi-head attention	8
MLP (feed-forward)	8
Residual connections	8
Think	8
Contagem de parametros	8
Output	8

Exercício 5: Training	10
Objetivo	10
Implementação.....	10
Cross-entropy loss.....	10
SGD.....	10
Adam.....	10
Resultados do treino	10
Reflect	11
Exercício 6: Generation and Experiments.....	12
Objetivo	12
Part A: Geracao com temperatura.....	12
Nomes gerados (temp=0.5)	12
Experiência 1: Temperatura	12
Predict.....	12
Observe.....	12
Explain.....	12
Experiência 2: Sem residual connections.....	12
Predict.....	12
Observe.....	13
Explain.....	13
Experiência 3: Sem RMSNorm.....	13
Predict.....	13
Observe.....	13
Explain.....	13
Experiência 4: 1 head vs. 4 heads.....	13
Predict.....	13
Observe.....	13
Explain.....	13
Experiência 5: Context window curta (block_size=4).....	14
Predict.....	14
Observe.....	14
Explain.....	14

Exercício 1: The Autograd Engine

Objetivo

Construir um motor de diferenciação automática (autograd). A classe Value regista cada operação e consegue percorrer o grafo computacional de trás para a frente (backward pass) para calcular os gradientes via chain rule. Além disso, é necessário implementar um tokenizer de caracteres para converter texto em sequências de inteiros.

Implementação

Operações aritméticas (add, mul)

Cada operação retorna um novo Value com os filhos e os gradientes locais (derivadas parciais). Para a adição, $d(a+b)/da = 1$ e $d(a+b)/db = 1$. Para a multiplicação, $d(a*b)/da = b$ e $d(a*b)/db = a$. Valores plain (int/float) são automaticamente convertidos em Value.

Operações unarias (pow, log, exp, relu)

Cada operação unaria tem um unico filho (self) e o gradiente local e a derivada correspondente: $\text{pow} \rightarrow n*x^{(n-1)}$, $\text{log} \rightarrow 1/x$, $\text{exp} \rightarrow e^x$, $\text{relu} \rightarrow 1$ se $x>0$, 0 caso contrario.

Backward pass

O backward constrói uma ordenação topológica do grafo via DFS (filhos antes dos pais), depois percorre em ordem inversa propagando gradientes. Usa-se += e não = porque um nó que alimenta duas operações recebe contribuições de gradiente de ambos os caminhos.

Think: $c = a*b + a$ ($a=2$, $b=3$)

Valor de $c = 2*3 + 2 = 8$. O no "a" participa em duas operações (multiplicação e adição), por isso $dc/da = b + 1 = 4$ e $dc/db = a = 2$. Confirmado pelos testes.

Tokenizer

Vocabulário de 27 tokens: 26 letras (a-z) + BOS (Beginning of Sequence, id=26). `encode()` converte cada caractere no seu índice em uchars e `decode()` converte ids de volta para caracteres, ignorando o BOS.

Output

```
=====
Exercise 1: The Autograd Engine
=====

[pass] addition and multiplication forward
[pass] mixed operations (Value with plain numbers)
[pass] pow, log, exp, relu
[pass] exp(log(x)) round-trip
[pass] backward: dc/da=4, dc/db=2 (matches your prediction?)
[pass] numerical gradient check
[pass] tanh gradient
```

Part A complete.

[pass] vocabulary, encode, decode

Dataset: 32033 names, 27 tokens

Example: 'yuheng' -> [24, 20, 7, 4, 13, 6]

```
=====
Exercise 1 complete!
=====
```

Todos os testes passaram, incluindo a verificação numérica de gradientes (finite differences) que confirma que os gradientes analíticos estão corretos.

Exercício 2: Neural Network Primitives

Objetivo

Implementar três primitivas usadas em todo o Transformer: linear (multiplicação matriz-vetor), softmax (converte scores em distribuição de probabilidade) e rmsnorm (normaliza vetores para estabilizar o treino).

Implementação

Linear

Multiplicação matriz-vetor: cada elemento do output é o dot product de uma linha da matriz W com o vetor x . Para W de shape $[n_out, n_in]$ e x de length n_in , o output tem length n_out .

Softmax

Fórmula: $\text{softmax}(x_i) = \exp(x_i) / \sum(\exp(x_j))$. O output satisfaz duas propriedades: todos os valores são positivos e somam 1 (distribuição de probabilidade válida). Para estabilidade numérica, subtrai-se o máximo dos logits antes de aplicar exp. Isto evita overflow em $\exp(1000)$ sem alterar o resultado, porque o fator $e^{(-c)}$ cancela no numerador e denominador.

RMSNorm

Normaliza o vetor para que o root-mean-square seja 1: $\text{output}_i = x_i / \sqrt{\text{mean}(x_i^2) + \text{eps}}$. O epsilon ($1e-5$) evita divisão por zero. Ao contrário do LayerNorm, o RMSNorm não subtrai a média, apenas escala pela magnitude.

Output

```
=====
Exercise 2: Neural Network Primitives
=====

[pass] linear: correct output shape and values
[pass] softmax: sums to 1, all positive, order preserved
[pass] softmax: handles large values (numerical stability)
[pass] rmsnorm: output has unit RMS
[pass] gradients flow through all three functions

=====
Exercise 2 complete!
=====
```

Reflect

Subtrair o max funciona porque $e^{(x_i - c)} / \sum(e^{(x_j - c)}) = e^{(x_i)} / \sum(e^{(x_j)})$, o $e^{(-c)}$ cancela. Qualquer constante funciona matematicamente, mas o max garante expoentes ≤ 0 , evitando overflow. É um padrão geral (log-sum-exp trick) aplicável sempre que se trabalha com exponenciais.

Exercício 3: Self-Attention

Objetivo

Implementar o mecanismo de self-attention com uma única cabeça. Cada token produz um Query ("o que estou a procurar?"), uma Key ("o que contem?") e um Value ("que informação carrego?"). Os scores de atenção são calculados pela similaridade $Q \cdot K$, normalizados por softmax, e usados para pesar os Values.

Implementação

Projeções Q, K, V

O embedding do token ($\text{tok_emb} + \text{pos_emb}$, normalizado por rmsnorm) e projetado por três matrizes de pesos para obter q, k e v. As keys e values são acumuladas numa KV cache que cresce a cada token — esta é a "memória" do modelo.

Scores de atenção

Para cada posição passada t, o score é o dot product $q \cdot \text{keys}[t]$ dividido por $\sqrt{\text{head_dim}}$. A divisão por $\sqrt{\text{head_dim}}$ evita que os scores cresçam proporcionalmente a dimensão, o que saturaria o softmax e concentraria toda a atenção num único token.

Weighted sum

Os attention weights (softmax dos scores) são usados para fazer uma soma pesada dos value vectors, dando mais importância aos tokens com keys mais semelhantes ao query atual.

Think

O primeiro token (posição 0) só pode atender a si próprio, o softmax de um único valor dá sempre 1.0. O terceiro token (posição 2) atende a 3 tokens com pesos que dependem da similaridade $Q \cdot K$: maior similaridade = mais peso. Os pesos não são iguais.

Output

```
=====
Exercise 3: Self-Attention
=====

[pass] output has correct dimension
[pass] attention weights sum to 1.0
[pass] KV cache grows: 1, 2, 3 positions
[pass] first token: 100% attention on itself (as predicted?)
[pass] different tokens produce different outputs

=====
Exercise 3 complete!
=====
```

Exercício 4: The GPT Model

Objetivo

Montar o modelo GPT completo: multi-head attention, MLP com expansão 4x, residual connections e normalização. A mesma arquitetura (em escala toy) que o GPT-2/3/4.

Implementação

Multi-head attention

Com $n_embd=16$ e $n_head=4$, cada cabeça ve $head_dim=4$ dimensões. Para cada cabeça h , extrai-se a slice $[h*4 : (h+1)*4]$ de Q , K e V . Cada cabeça calcula a atenção independentemente no seu subespaço, depois os outputs são concatenados (16 dimensões no total) e projetados por $attn_wo$.

MLP (feed-forward)

Duas camadas lineares com ReLU entre elas: primeiro expande de n_embd para $4*n_embd$ (64), aplica ReLU, depois comprime de volta para n_embd (16). O MLP tem 2048 parâmetros, o dobro da atenção (1024). Esta proporção 2:1 MLP:attention mantém-se em modelos reais.

Residual connections

Cada sub-bloco (atenção e MLP) tem uma residual connection: $output = x + f(x)$. O gradiente de $x + f(x)$ em relação a x é $1 + f'(x)$, garantindo que o gradiente nunca desaparece completamente, cria uma "autoestrada" para os gradientes evoluírem até as primeiras camadas.

Think

Múltiplas cabeças permitem ao modelo capturar diferentes perspectivas em paralelo (ex: ler uma frase pela gramática vs. pelo significado vs. pelo tom emocional). A residual connection garante que o gradiente flui mesmo quando $f(x)$ contribui pouco, estabilizando o treino em redes profundas.

Contagem de parâmetros

wte: $27*16 = 432$, wpe: $16*16 = 256$, attn (Q,K,V,O): $4*16*16 = 1024$, mlp (fc1+fc2): $16*64 + 64*16 = 2048$, lm_head: $27*16 = 432$. Total: 4192 parametros.

Output

```
=====
Exercise 4: The GPT Model
=====

[pass] output has vocab_size (27) logits
[pass] 3-token sequence processes correctly
[pass] different inputs -> different outputs
[pass] parameter count = 4,192
[pass] gradients flow from output to embeddings
=====
```

Exercise 4 complete!

=====

Exercício 5: Training

Objetivo

Treinar o modelo: implementar a função de loss (cross-entropy), o SGD e o Adam, e observar o modelo a aprender padrões em nomes.

Implementação

Cross-entropy loss

Para cada posição na sequência, o modelo prevê uma distribuição de probabilidade sobre os 27 tokens. A loss é $-\log(p_{\text{correto}})$, a média sobre todas as posições. Com pesos aleatórios, $p \sim 1/27$ para cada token, logo a loss inicial esperada é $-\log(1/27) = 3.296$.

SGD

O otimizador mais simples: $p.data -= lr * p.grad$. Atualiza cada parâmetro na direção oposta ao gradiente, com um learning rate fixo igual para todos.

Adam

Mantem duas médias moveis por parâmetro: m (momentum, gradiente suavizado) e v (RMS, gradiente ao quadrado suavizado). Aplica bias correction para os primeiros steps e usa learning rate adaptativo por parâmetro: $lr * m_hat / (\sqrt{v_hat} + \epsilon)$. Converge muito mais rápido que SGD.

Resultados do treino

```
--- Phase 1: SGD (50 steps) ---
step  1 | loss 3.6607
step 10 | loss 3.2944
step 20 | loss 3.0983
step 30 | loss 3.3624
step 40 | loss 3.2190
step 50 | loss 3.2470

Initial loss: 3.6607 (expected ~3.30)
```

```
--- Phase 2: Adam (200 steps) ---
step  1 | loss 3.4697
step 20 | loss 2.7294
step 40 | loss 2.6779
step 60 | loss 2.4926
step 80 | loss 2.6632
step 100 | loss 2.5136
step 120 | loss 2.0913
step 140 | loss 2.4232
step 160 | loss 2.2718
step 180 | loss 2.7223
step 200 | loss 1.8090
```

```
--- Results ---
```

```
SGD after 50 steps: 3.2470
Adam after 200 steps: 1.8090
[pass] loss decreased during training
[pass] final loss < 2.5 (good convergence)
```

Reflect

A loss inicial (3.66) está próxima do esperado (~3.30). Adam converge muito mais rápido que SGD: em 200 steps chega a 1.81, enquanto SGD mal desceu de 3.66 para 3.25 em 50 steps. Adam adapta o learning rate por parâmetro e usa momentum, evitando oscilações.

O linear decay do learning rate ajuda porque no fim do treino o modelo está próximo do mínimo. Um lr alto faria o modelo saltar por cima do mínimo repetidamente sem conseguir convergir. Reduzir o lr gradualmente permite passos cada vez mais finos, aproximando-se do mínimo sem o ultrapassar.

O modelo não consegue chegar a loss 0 porque é demasiado pequeno (16 dimensões, 1 camada, 4192 parâmetros) para modelar todas as dependências nos 32000 nomes. Para melhorar seria necessário mais camadas, mais dimensões e mais tempo de treino.

Exercício 6: Generation and Experiments

Objetivo

Implementar a geração de texto com controlo de temperatura e realizar experiências de ablação para entender porque é que cada componente do Transformer importa.

Part A: Geração com temperatura

A geração funciona token a token: o modelo recebe o token anterior, produz logits, que são divididos pela temperatura antes de aplicar softmax. Amostra-se o próximo token com `random.choices` segundo a distribuição resultante. Se o token amostrado for BOS, o nome termina.

Nomes gerados (temp=0.5)

- | | |
|-----------|-----------|
| 1. kamar | 6. mayli |
| 2. ogdan | 7. konnan |
| 3. akele | 8. alame |
| 4. marien | 9. elatin |
| 5. cahli | 10. mamyn |

[pass] all generated names use valid characters

Experiência 1: Temperatura

Predict

temp=0.1: logits divididos por 0.1 ficam muito maiores, softmax amplifica as diferenças, o token mais provável fica com probabilidade quase 1. Nomes repetitivos e "seguros". temp=0.5: ainda favorece os tokens mais prováveis mas com alguma variedade. temp=1.0: sem alteração, comportamento normal. temp=2.0: logits divididos por 2 ficam menores, softmax nivela as diferenças, escolha quase aleatória. Nomes estranhos.

Observe

temp=0.1: aman, aran, aman, aman, aman
temp=0.5: kamar, ogdan, akele, marien, cahli
temp=1.0: maker, tamainan, kanta, maezeei, lori
temp=2.0: naiest, blaglahyrfsyawri, cyibcys, rl, imuoyovaihahaipi

Explain

Com temp=0.1, o modelo repete quase sempre "aman" — o token mais provável domina completamente. Com temp=0.5, os nomes são variados e parecem reais. Com temp=1.0, começam a aparecer nomes mais longos e menos comuns. Com temp=2.0, os nomes são puro ruído, a distribuição é quase uniforme o que origina nomes com caracteres aleatórios.

Experiência 2: Sem residual connections

Predict

Sem residual connections o gradiente pode desaparecer nas camadas mais profundas. O modelo vai aprender mais devagar ou não convergir de todo.

Observe

Baseline final loss: 2.0386
No-residual final loss: 2.7781

Explain

Sem residual connections a loss ficou em 2.78 vs 2.04 com residual. O modelo ainda aprendeu (desceu do valor inicial), mas convergiu pior e para um mínimo mais alto. Sem o $x + f(x)$, os gradientes só têm o caminho mais "longo" para fluir, atenuando-se camada a camada. Com 1 camada o efeito é moderado; com mais camadas seria muito mais grave.

Experiência 3: Sem RMSNorm

Predict

Sem RMSNorm os valores podem crescer sem controle camada a camada, os gradientes explodem e o treino provavelmente crasha com overflow.

Observe

Baseline final loss: 2.0386
No-norm final loss: 2.7955

Explain

Não crashou. O modelo aprendeu, mas convergiu pior (2.80 vs 2.04). O treino ficou mais instável (a loss a oscilar mais) mas não deu crash. O modelo é pequeno demais para os valores crescerem o suficiente para causar overflow. Em modelos maiores a normalização é essencial para a estabilidade.

Experiência 4: 1 head vs. 4 heads

Predict

4 heads aprende melhor. Cada cabeça foca-se em padrões e perspectivas diferentes do input em paralelo (subespaços distintos do embedding). Com 1 cabeça, o modelo tenta capturar tudo com uma única distribuição de atenção, limitando a expressividade.

Observe

1 head final loss: 2.1528
4 heads final loss: 2.0386

Explain

4 heads convergiram para uma loss mais baixa que 1 head, confirmando a previsão. Apesar de cada head ter `head_dim=4` (vs 16 com 1 head), a diversidade de padrões de atenção que podem ser aprendidos em paralelo compensa a menor capacidade individual por head. Com 1 head, o modelo é forçado a comprimir toda a informação contextual numa única matriz de atenção, enquanto 4 heads permitem especializações diferentes (e.g., uma head aprende dependências de curto alcance, outra de longo alcance).

Experiência 5: Context window curta (block_size=4)

Predict

Com block_size=4 o modelo só vê os últimos 4 tokens, por isso os nomes gerados com mais de 4 caracteres vão ficar incoerentes ou simplesmente não vão existir, o modelo nunca aprendeu a gerar além dessa posição.

Observe

```
block_size=4 final loss: 2.7200  
block_size=16 final loss: 2.0386
```

Names with block_size=4:

```
kale (length 4)  
mava (length 4)  
jali (length 4)  
alin (length 4)  
bria (length 4)
```

Explain

Todos os nomes gerados têm exatamente 4 caracteres, o modelo nunca treinou em sequências mais longas que 4 posições, por isso não aprendeu a continuar a gerar além disso. Os nomes parecem plausíveis nos primeiros caracteres, mas são todos truncados.